

DMD su Raspberry Pi — Procedura di ricostruzione da zero

Pannelli LED P2.5 128x64 con chip **FM6373** (S-PWM), 2 moduli in cascata → 256x64 px.

Modelli coperti da questo documento: **Pi Zero W**, **Pi Zero 2 W**, **Pi 3**, **Pi 4**.

Punto di partenza: microSD già scritta con Raspberry Pi OS Lite (Legacy, 64-bit — per la Pi Zero W usare la versione **32-bit**), con WiFi, utente e SSH già configurati tramite Raspberry Pi Imager.

Utente di riferimento in questo documento: `gillo`, hostname `dmdpi`.

0. Perché questa procedura esiste

I pannelli montano il driver **FM6373**, un chip S-PWM con framebuffer integrato. Non è supportato da:

- ESP32-HUB75-MatrixPanel-DMA (e quindi **non da ZeDMD**)
- la libreria ufficiale `hzeller/rpi-rgb-led-matrix`

È supportato **solo** dal fork sperimentale `kingdo9/rpi-rgb-led-matrix_pwm_experiment`, che funziona **esclusivamente su Raspberry Pi** (scrive direttamente sui registri Broadcom / RP1: Orange Pi, Radxa, Banana Pi e simili sono esclusi). Inoltre i pannelli espongono solo le linee di indirizzo **A**, **B**, **C** (D ed E sono serigrafate NC) e richiedono un profilo di registro specifico, individuato sperimentalmente.

1. Scelta del modello

Modello	Core	--led- slowdown- gpio	isolcpus	Giudizio
Pi Zero W (v1.1)	1 × ARM11	1 (provare 2)	non applicabile	Funziona, ma senza core dedicato. Adatto a contenuti statici e slideshow; margine ridotto per streaming DMD + web + video insieme.
Pi Zero 2 W	4 × A53	3	<code>isolcpus=3</code>	Ottimo compromesso ingombro/prestazioni. Solo WiFi 2.4 GHz.
Pi 3 (3A+/3B/3B+)	4 × A53	3	<code>isolcpus=3</code>	Come la Zero 2 W ma con 1 GB di RAM e più porte. Facile da reperire.
Pi 4	4 × A72	5	<code>isolcpus=3</code>	Il più capace: margine abbondante per tutte le funzioni insieme, WiFi 5 GHz ed ethernet. Scalda: prevedere dissipatore. Richiede alimentatore USB-C 5V/3A serio.

Il **cablaggio è identico su tutti i modelli**: l'header a 40 pin ha la stessa disposizione. Sulle Zero l'header potrebbe non essere saldato di fabbrica.

Nel resto del documento i comandi usano la variabile `$SLOW`. Impostala **all'inizio di ogni sessione SSH** con il valore della tabella qui sopra, ad esempio per una Zero 2 W o una Pi 3:

```
export SLOW=3
```

Per la Pi Zero W sarà `export SLOW=1`, per la Pi 4 `export SLOW=5`.

2. Primo accesso

Inserire la microSD e alimentare il Pi:

- Zero W / Zero 2 W: porta micro-USB marcata **PWR**
- Pi 3: micro-USB, alimentatore 5V/2.5A
- Pi 4: USB-C, alimentatore 5V/3A

Attendere 2-3 minuti (primo boot + riavvio automatico), poi dal Mac:

```
ssh gillo@dmdpi.local
```

3. Preparazione del sistema

Aggiornamento e strumenti di compilazione:

```
sudo apt update && sudo apt full-upgrade -y  
sudo apt install -y git build-essential
```

3.1 Disattivare l'audio integrato (obbligatorio, tutti i modelli)

L'audio onboard usa lo stesso hardware PWM necessario a pilotare i pannelli.

```
sudo sed -i 's/^dtparam=audio=on/dtparam=audio=off/' /boot/firmware/config.txt  
echo "blacklist snd_bcm2835" | sudo tee /etc/modprobe.d/blacklist-audio.conf
```

3.2 Riservare un core della CPU al refresh

Solo su Pi Zero 2 W, Pi 3 e Pi 4. Sulla Pi Zero W questo passo va **saltato**: ha un solo core e isolarlo bloccherebbe il sistema.

```
sudo sed -i 's/$/ isolcpus=3/' /boot/firmware/cmdline.txt
```

3.3 Verifica e riavvio

```
grep audio /boot/firmware/config.txt  
cat /boot/firmware/cmdline.txt  
sudo reboot
```

`config.txt` deve contenere `dtoverlay=audio=off`. Su modelli quad-core `cmdline.txt` deve terminare con `isolcpus=3`, su **una sola riga**.

4. Installazione della libreria

```
git clone https://github.com/kingdo9/rpi-rgb-led-matrix_pwm_experiment.git
cd rpi-rgb-led-matrix_pwm_experiment
make
cd examples-api-use
make
```

Tempi indicativi: pochi minuti su Pi 4 e Pi 3, qualche minuto in più sulla Zero 2 W, sensibilmente di più sulla Zero W.

5. Cablaggio

Tutto rigorosamente a dispositivi spenti. Identico su tutti i modelli.

5.1 Segnali dati: connettore HUB75 del pannello → GPIO del Pi

Mappatura "regular" della libreria hzeller. Pin fisici contati sull'header a 40 poli (pin 1 = angolo lato microSD, fila interna; dispari sulla fila interna, pari su quella esterna).

Segnale pannello	GPIO (BCM)	Pin fisico Pi
R1	GPIO 11	23
G1	GPIO 27	13
B1	GPIO 7	26
R2	GPIO 8	24
G2	GPIO 9	21
B2	GPIO 10	19
A	GPIO 22	15
B	GPIO 23	16
C	GPIO 24	18
CLK	GPIO 17	11
LAT	GPIO 4	7
OE	GPIO 18	12
GND	—	6 e 14

D ed E non si collegano (NC sul pannello). I GPIO che la libreria vi assocerebbe (25 e 15) restano liberi.

5.2 Cascata dei due pannelli

Pi → connettore **JIN** (ingresso) del primo pannello; **JOUT** (uscita) del primo → **JIN** del secondo.
Ogni pannello ha la propria alimentazione 5V.

5.3 Alimentazione — tre regole non negoziabili

1. I pannelli si alimentano da un **alimentatore 5V esterno** tramite i propri cavi di potenza ($\approx 4A$ per pannello), mai dal Pi.
2. Il Pi si alimenta dal proprio alimentatore (vedi §2 per il modello).
3. **Le masse devono essere in comune:** il GND dell'alimentatore pannelli e il GND del Pi (pin 6/14 → GND del connettore HUB75) devono essere elettricamente uniti.

I segnali del Pi sono a 3.3V mentre il pannello ne attende 5V: nella pratica funziona direttamente. Un level-shifter serve solo se compaiono sfarfallii persistenti.

6. Configurazione funzionante

Questi parametri sono il risultato della fase diagnostica (vedi §9). Vanno usati **tutti insieme**.

Parametro	Valore	Ruolo
--led-rows	64	righe del pannello
--led-cols	128	colonne del singolo pannello
--led-chain	2	numero di pannelli in cascata (1 per test singolo)
--led-panel-type	fm6373	driver S-PWM
--led-spwm-row-addr-type	1	indirizzamento righe a shift register
--led-spwm-scan	64	critico: con 32 l'immagine si duplica verticalmente
--led-spwm-register-config	2	critico: profilo di registro del chip
--led-no-drop-privs	—	critico: senza, il catalogo profili non è leggibile
--led-slowdown-gpio	\$SLOW	dipende dal modello (\$1)
--led-limit-refresh	60	stabilità, riduce artefatti
--led-brightness	50	regolabile 0-100

Variabile d'ambiente obbligatoria:

```
SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register_test/data
```

I profili 2, 6, 42, 68 danno risultato equivalente (tutti Scan_64, sorgente P2.5 - FM6373 - DP32020B - 1/64). In caso di problemi con uno, provare gli altri tre.

7. Script di avvio

Sostituire il valore di `SLOW` nella seconda riga secondo il modello in uso (§1).

```
cat > ~/dmd.sh << 'EOF'
#!/bin/bash
SLOW=3
export SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register-test/data
BIN=~ /rpi-rgb-led-matrix_pwm_experiment/examples-api-use
OPTS="--led-no-drop-privs --led-rows=64 --led-cols=128 --led-chain=2 --led-panel-type=fm6373 --led-
spwm-row-addr-type=1 --led-spwm-scan=64 --led-spwm-register-config=2 --led-slowdown-gpio=$SLOW --led-
limit-refresh=60 --led-brightness=50"
sudo -E $BIN/demo -D0 $OPTS
EOF
chmod +x ~/dmd.sh
```

Avvio: `~/dmd.sh`

8. Verifica del funzionamento

Quadrato rotante (demo D0)

```
~/dmd.sh
```

Atteso: un quadrato intero che ruota e rimbalza, senza duplicazioni né pezzi mancanti.

Testo scorrevole

```
export SLOW=3
sudo SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/register-test/data \
~/rpi-rgb-led-matrix_pwm_experiment/examples-api-use/scrolling-text-example \
-f ~/rpi-rgb-led-matrix_pwm_experiment/fonts/9x18.bdf -s 2 -y 30 \
--led-no-drop-privs --led-rows=64 --led-cols=128 --led-chain=2 \
--led-panel-type=fm6373 --led-spwm-row-addr-type=1 --led-spwm-scan=64 \
--led-spwm-register-config=2 --led-slowdown-gpio=$SLOW --led-limit-refresh=60 \
--led-brightness=50 "TEST DMD"
```

Cosa osservare per modello

Su Pi 4, Pi 3 e Zero 2 W l'immagine deve risultare stabile. Sulla **Pi Zero W** è normale dover cercare il valore migliore tra `--led-slowdown-gpio=1` e `=2`, ed è possibile che restino artefatti sporadici: in quel caso ridurre il carico di sistema o valutare un modello quad-core.

9. Programma diagnostico `canvas-map`

Utile se in futuro cambiano pannelli o configurazione: colora il canvas a bande per capire come i pixel logici finiscono su quelli fisici.

Creare `~/canvas-map.cc` con `nano` (l'incollaggio di blocchi lunghi via heredoc può corrompersi):

```

#include "led-matrix.h"
#include <signal.h>
#include <unistd.h>
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

using namespace rgb_matrix;

volatile bool run_flag = true;
static void stop(int) { run_flag = false; }

int main(int argc, char **argv) {
    RGBMatrix::Options opt;
    rgb_matrix::RuntimeOptions rt;
    if (!ParseOptionsFromFlags(&argc, &argv, &opt, &rt)) return 1;
    const char *mode = (argc > 1) ? argv[1] : "cols";
    RGBMatrix *m = RGBMatrix::CreateFromOptions(opt, rt);
    if (m == NULL) return 1;
    FrameCanvas *c = m->CreateFrameCanvas();
    int W = c->width(), H = c->height();
    printf("Canvas: %dx%d, mode=%s\n", W, H, mode);
    if (strcmp(mode, "axes") == 0) {
        for (int x = 0; x < W; x++) c->SetPixel(x, 0, 200, 0, 0);
        for (int y = 0; y < H; y++) { c->SetPixel(0, y, 0, 200, 0); c->SetPixel(1, y, 0, 200, 0); }
        c->SetPixel(0, 0, 255, 255, 255);
    } else if (strcmp(mode, "pair") == 0 && argc > 3) {
        int px = atoi(argv[2]);
        int py = atoi(argv[3]);
        c->SetPixel(px, py, 255, 0, 0);
        c->SetPixel(px + 1, py, 0, 255, 0);
        c->SetPixel(px + 2, py, 0, 0, 255);
    } else {
        for (int x = 0; x < W; x++) {
            for (int y = 0; y < H; y++) {
                uint8_t r = 0, g = 0, b = 0;
                int band;
                if (strcmp(mode, "fine") == 0) {
                    band = (x / 8) % 3;
                    if (band == 0) r = 180;
                    else if (band == 1) g = 180;
                    else b = 180;
                } else {
                    if (strcmp(mode, "cols") == 0) band = x * 4 / W;
                    else band = y * 4 / H;
                    if (band == 0) r = 180;
                    else if (band == 1) g = 180;
                    else if (band == 2) b = 180;
                    else { r = 180; g = 180; }
                }
                c->SetPixel(x, y, r, g, b);
            }
        }
    }
    c = m->SwapOnVSync(c);
    signal(SIGINT, stop);
    while (run_flag) usleep(100000);
    m->Clear();
    delete m;
    return 0;
}

```

Compilazione:

```
g++ -O2 -o ~/canvas-map ~/canvas-map.cc \  
-I ~/rpi-rgb-led-matrix_pwm_experiment/include \  
-L ~/rpi-rgb-led-matrix_pwm_experiment/lib \  
-lrgbmatrix -lrt -lm -lpthread
```

Modalità disponibili (primo argomento):

- `cols` — 4 bande verticali (rosso/verde/blu/giallo) per mappare le colonne
- `rows` — 4 bande orizzontali per mappare le righe
- `fine` — strisce verticali da 8 px per misurare il fattore di scala
- `axes` — riga 0 in rosso, colonne 0-1 in verde: rivela l'orientamento
- `pair X Y` — tre pixel adiacenti R/G/B: rivela duplicazioni e sfasamenti

Esempio:

```
export SLOW=3  
sudo SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/registertest/data \  
~/canvas-map rows --led-no-drop-privs --led-rows=64 --led-cols=128 --led-chain=1 \  
--led-panel-type=fm6373 --led-spwm-row-addr-type=1 --led-spwm-scan=64 \  
--led-spwm-register-config=2 --led-slowdown-gpio=$SLOW --led-limit-refresh=60 --led-brightness=50
```

Configurazione corretta = quattro bande di quattro colori diversi, ciascuna una sola volta.

10. Ricerca del profilo di registro (se cambiano i pannelli)

Demo interattiva che scorre i 77 profili disponibili per FM6373:

```
export SLOW=3  
sudo SPWM_PROFILE_DIR=/home/gillo/rpi-rgb-led-matrix_pwm_experiment/lib/spwm/registertest/data \  
~/rpi-rgb-led-matrix_pwm_experiment/examples-api-use/demo -D15 --led-no-drop-privs \  
--led-rows=64 --led-cols=128 --led-chain=1 --led-panel-type=fm6373 \  
--led-spwm-row-addr-type=1 --led-spwm-scan=64 --led-slowdown-gpio=$SLOW \  
--led-limit-refresh=60 --led-brightness=50
```

Frecce **sinistra/destra** per scorrere, **M** per marcare un profilo come buono. Il numero mostrato si riusa in `--led-spwm-register-config=N`.

11. Risoluzione problemi

Sintomo	Causa	Rimedio
unable to open register-profile catalog	la libreria abbandona i privilegi di root e daemon non può leggere /home/gillo	aggiungere <code>--led-no-drop-privs</code>
unable to locate fm6373.profiles	variabile d'ambiente assente	impostare <code>SPWM_PROFILE_DIR</code> (con <code>sudo -E</code> se esportata)
Immagine duplicata verticalmente	<code>--led-spwm-scan=32</code>	usare <code>--led-spwm-scan=64</code>
Barre colorate scombinare, immagine spezzata	profilo di registro sbagliato	<code>--led-spwm-register-config=2</code> (o 6/42/68)
Pannello nero dopo test falliti	i chip S-PWM restano in stato inconsistente	power-cycle del pannello: staccare il suo 5V per 10 secondi (il Pi può restare acceso)
Solo metà pannello acceso	<code>--led-spwm-data-layout</code> 4 o 5	non usare <code>--led-spwm-data-layout</code> (default 0)
demo: command not found	percorso sbagliato	usare il percorso completo o <code>cd ~/rpi-rgb-led-matrix_pwm_experiment/examples-api-use</code>
Sfarfallio, righe bianche sporadiche	valore di slowdown errato per il modello	verificare <code>\$SLOW</code> secondo §1; se persiste, <code>--led-limit-refresh=60</code> e tuning secondo <code>spwm.md</code> del fork
Immagine instabile su Pi 4	slowdown troppo basso	il Pi 4 richiede 5, non 3
Comportamenti erratici su Pi 4	alimentatore insufficiente	usare un USB-C 5V/3A di qualità

Regola d'oro durante i test: power-cycle del pannello tra un tentativo e l'altro. I chip S-PWM memorizzano la configurazione nei registri interni e una configurazione sbagliata può bloccarli fino allo spegnimento.

Non spegnere mai il Pi togliendo corrente: `sudo poweroff`, attendere che il LED verde smetta di lampeggiare, poi staccare.

12. Specifiche dell'hardware

Pannelli: P2.5 indoor, 128x64 px, 320x160 mm, driver **FM6373** (S-PWM), row driver DP32020B, linee di indirizzo A/B/C (D ed E = NC), interfaccia HUB75.

Risoluzione finale: 256x64 px con i due pannelli in cascata.

Controller: qualsiasi Raspberry Pi tra quelli elencati in §1. Altre schede a scheda singola (Orange Pi, Radxa, Banana Pi) **non sono utilizzabili:** la libreria accede direttamente ai registri hardware

13. Riferimenti

- Fork con supporto S-PWM: https://github.com/kingdo9/rpi-rgb-led-matrix_pwm_experiment
- Guida al tuning S-PWM: `spwm.md` nel repository
- Libreria originale: <https://github.com/hzeller/rpi-rgb-led-matrix>
- Discussione sui chip S-PWM: <https://github.com/hzeller/rpi-rgb-led-matrix/issues/1866>